

A linear bound disproves conjecture 00000003943

ziangni-sys

Prepared with OpenAI Codex

4 October 2026

Abstract

The classical augmenting-path algorithm for maximum-cardinality matching uses at most $\lfloor n/2 \rfloor$ augmenting paths on an n -vertex graph. Consequently its worst-case number of augmenting paths cannot be $\Theta(n^2 \log n)$, as conjectured. The argument applies to general graphs, arbitrary initial matchings, and arbitrary choices of augmenting paths. A self-contained Lean 4 project verifies the matching bound, the actual path toggle, and the asymptotic contradiction.

1 Original claim and the quantity being counted

The English statement is:

Definition: The augmenting path algorithm is the classical iterative method for matching in general graphs. Conjecture: In the worst case the total number of augmenting paths is $\Theta(n^2 \log n)$, and the ratio between the upper and lower bounds is at most 2. (tight augmenting path bound)

We use $n = |V(G)|$, the conventional input vertex count. The source identifies the *classical iterative method*: initialize a matching M , find an M -augmenting path P , replace M by $M \triangle E(P)$, and repeat until no such path exists. Thus the total number of augmenting paths in this algorithm is the number of paths selected and used for augmentation. This is the standard loop described in matching lectures and texts; see the university references below.

This statement does not specify an enumeration of *all available* augmenting paths or a count of failed search attempts, examined edges, blossom operations, or running-time instructions. Those are different quantities. In particular, the result here concerns the stated iterative matching method and its augmentations. Interpreting the conjecture as a different unspecified search-operation count would require a new definition. The disproof does not silently identify that count with augmentations.

2 Definitions and the size invariant

Let $G = (V, E)$ be a finite simple undirected graph. A matching $M \subseteq E$ is a set of pairwise vertex-disjoint edges. An M -augmenting path is a simple path

$$P = (v_0, v_1, \dots, v_{2r+1})$$

whose endpoints are unmatched and whose edges alternate between outside and inside M , starting and ending outside M . Write $e_i = \{v_{i-1}, v_i\}$ for $1 \leq i \leq 2r+1$. Then the r even-indexed edges belong to M and the $r+1$ odd-indexed edges do not.

Lemma 1. *Augmentation along P yields a matching $M' = M \triangle E(P)$ with $|M'| = |M| + 1$.*

Proof. The update removes the r matching edges of the path and inserts its $r+1$ nonmatching edges, so

$$|M'| = |M| - r + (r + 1) = |M| + 1.$$

The inserted edges pair the vertices $(v_0, v_1), (v_2, v_3), \dots, (v_{2r}, v_{2r+1})$. They have disjoint endpoints because the path is simple. Every interior path vertex loses its previous matching edge, and the endpoints had no matching edge. Edges of M outside the path therefore share no endpoint with an inserted edge. This proves that M' is a matching. $\square$

Theorem 1. *For any run with k augmentations from M_0 to M_k on an n -vertex graph,*

$$|M_k| = |M_0| + k, \quad k \leq \lfloor n/2 \rfloor - |M_0| \leq \lfloor n/2 \rfloor.$$

Proof. The first identity follows by induction using the lemma. Since matching edges use two distinct vertices each, $2|M_k| \leq n$. Substitute the identity and rearrange. The proof also bounds every partial run, so it applies independently of the search procedure, tie breaking, blossoms, or termination criterion. $\square$

The bound is tight when starting from the empty matching: a graph consisting of $\lfloor n/2 \rfloor$ disjoint edges and, for odd n , one isolated vertex requires exactly one augmentation per edge. Every available augmenting path in that graph is a single edge. Hence the worst-case number is exactly $\lfloor n/2 \rfloor$, in particular $\Theta(n)$.

3 Contradiction with the claimed growth

Suppose the asserted $\Theta(n^2 \log n)$ growth held. Its lower bound would give $c > 0$ and N such that, for all sufficiently large n , some run has

$$cn^2 \log n \leq k \leq n/2.$$

For n with $\log n \geq 1$ and $n > 1/(2c)$, this is impossible. The purported second clause about the ratio of bounds cannot rescue a false first clause.

Lean uses the equivalent asymptotic scale $n^2 \lfloor \log_2 n \rfloor$ and an integer reciprocal lower-bound constant $C > 0$. These changes preserve the lower-bound question: any positive real constant can be decreased to $1/C$, and changing a fixed logarithm base or taking its floor changes the scale by bounded constant factors for sufficiently large n . The formal negation assumes

$$n^2 \lfloor \log_2 n \rfloor \leq Ck$$

for an admissible run at every $n \geq N$, chooses $n = C + N + 2$, and derives

$$n^2 \leq n^2 \lfloor \log_2 n \rfloor \leq Ck \leq Cn < n^2.$$

This is an asymptotic contradiction, not an isolated numerical counterexample.

4 Formalization and reproducibility

The Lean project imports only `Std` and pins Lean 4.19.0. `Graph` records a vertex bound and a symmetric irreflexive adjacency relation. `Matching` records canonical graph edges, disjoint endpoints, and vertex bounds. The theorem `matching_bound` proves the $2|M| \leq n$ capacity bound from these definitions, using a duplicate-free endpoint list contained in $\{0, \dots, n-1\}$.

`AugmentingPath` contains the actual simple vertex list, even positive vertex count, alternating matching membership, graph membership, and unmatched endpoints. `freeEdges` and

`matchedEdges` extract the two edge classes from consecutive vertices; `toggle` removes the latter and appends the former. `augmentation_size` proves its exact cardinality change. An explicit duplicate-free condition on the removed edges is included; it is redundant for a simple path, and ensures the removal lemma applies without relying on an unstated list convention.

`Run` records the actual matchings at every step, the selected augmenting path, and equality of the next edge list to the toggle. Thus its records include the matching invariants at every step. The formal proof does not separately construct the next matching from an arbitrary path; the elementary validity proof is the lemma above. Every run of the classical algorithm supplies precisely such records. `run_size`, `run_bound`, and the final disproof theorem establish the size identity, linear bound, and negation of the stated asymptotic lower law. No condition about finding shortest paths or starting from an empty matching is assumed.

The theorem audits report only Lean’s standard logical axioms:

`propext`, `Classical.choice`, and `Quot.sound`.

There are no additional axioms, admitted proofs, or `native_decide`. Run lake build and lake env lean Main.lean in lean/. The independent script `verify.py` exhaustively checks every labeled simple graph through five vertices, all its matchings, and all simple augmenting paths, including that every toggle remains a matching and increases its size by one. It also checks the tight disjoint-edge family. Its verification is supplementary: the universal disproof follows from the proofs above and the Lean theorems.

References

- [1] Princeton University, *COS 423: Graph Matching*, lecture slides. <https://www.cs.princeton.edu/courses/archive/spr11/cos423/Lectures/GraphMatching.pdf>.
- [2] N. Zeh, Dalhousie University, *Algorithms II: Augmenting Paths*. https://web.cs.dal.ca/~nzeh/Teaching/4113/book/matching/bipartite_maximum_matching/augmenting_paths.html.